COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-CPE21S3 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	7/16/2026	**DATE SUBMITTED:**	7/20/2026
NAME:	Gabriel Immanuel R. Serrano	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

Screenshot

Act2_ILOA.cpp > ...

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName ="John Doe", int newAge=18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };

        //deconstructor
        ~Student(){
            std::cout << "Deconstructor Called." << std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        //Display Attributes
        void printDetails(){
            std::cout << this->studentName << " " << this->studentAge << std::endl;
        }
};


int main(){
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

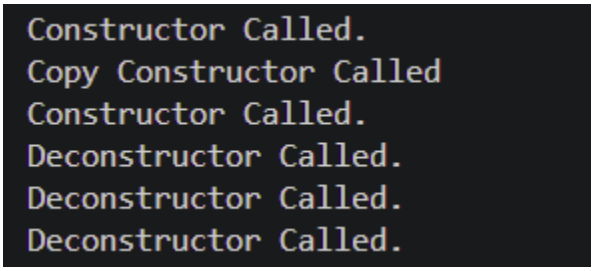
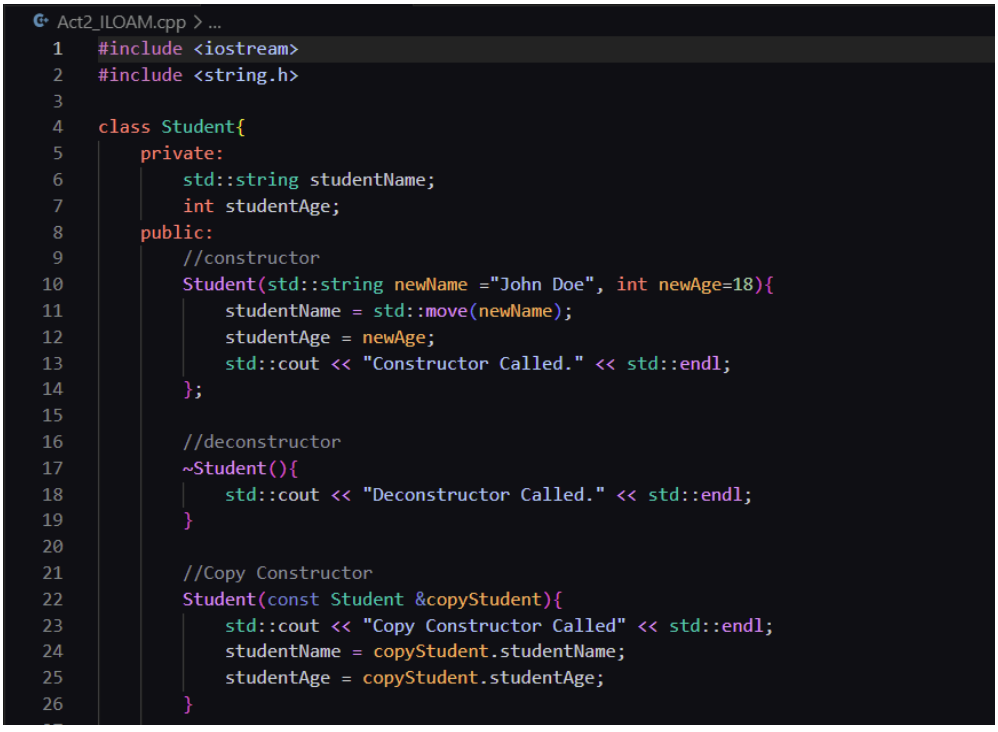
	
Observation	The code creates three Student objects to demonstrate the constructor, copy constructor, and copy assignment operator. First, student1 is created using the constructor with the values "Roman" and 28. Inside the constructor, std::move transfers the value of the parameter newName into the studentName variable, while studentAge is assigned the value 28. Next, student2 is created using the copy constructor, which copies the name and age stored in student1, so both objects contain the same information even though it shows on the string that student2 is “ ”. After that, student3 is created using the default constructor, which initializes it with the default values "John Doe" and 18. The copy assignment operator is then used to assign student2 to student3, which replaces the default values with the copied values. Lastly, the destructor was called at the end of the program to release the memory used by each object.

Table 2-1. Initial Driver Program

Screenshot	

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName ="John Doe", int newAge=18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };

        //deconstructor
        ~Student(){
            std::cout << "Deconstructor Called." << std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }
```

```
27
28          //Display Attributes
29          void printDetails(){
30              std::cout << this->studentName << " " << this->studentAge << std::endl;
31          }
32  };
33
34
35  int main(){
36      const size_t j = 5;
37
38      Student studentList[j] = {};
39      std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
40      int ageList[j] = {15, 16, 18, 19, 16};
41
42      return 0;
43  }
```

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
```

Observation	The code created an array of Student objects with a constant size of 5. When the array was initialized, each object was automatically created using the default constructor, so the constructor was called five times. As a result, every object in the array contained the default values of the constructor which is “John Doe” and 18. The program also created the namesList and ageList arrays, which have the names and ages of 5 students already defined. However, these names and ages didn’t replace the default values in the studentList array because they were never assigned to the objects. Lastly, the program deconstructs all the objects on the studentList.

Table 2-2. Modified Driver Program with Student Lists

Loop A	`for(int i = 0; i < j; i++){ // loop A`
	`    Student *ptr = new Student(namesList[i], ageList[i]);`
	`    studentList[i] = *ptr;`
	`}`

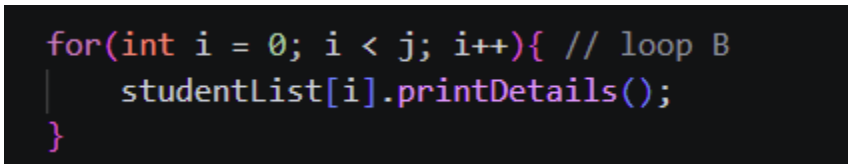
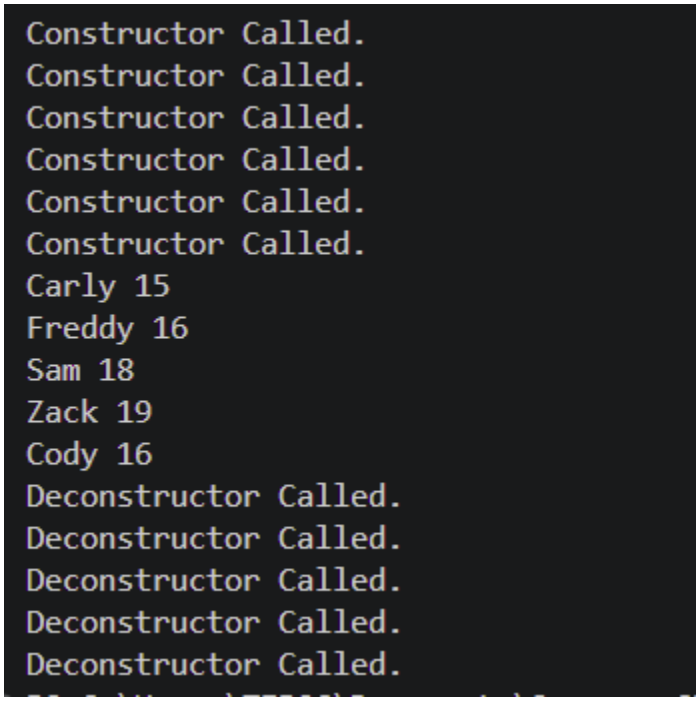
Observation	This for loop replaced all the “John Doe” and the age 18 on the studentList array, this allows you to modify the studentList arrays. With every iteration of the for loop it points at the address on the nameList which stores the name of the student then it points on the student object which is the array now the address that we got on the nameList is going to replace the studentName on that object, it will loop and loop until the i is greater than 5.
Loop B	
Observation	This for loop prints the names and ages of the students stored in studentList. The -> operator is used to access a member of an object through a pointer. It allows the program to access the studentName and studentAge members of each Student object and print their values.
Output	
Observation	The output displayed how the program worked by showing that the constructor was called each time a Student object was created, this happened when we have the “John Doe” and 18 on the array. It also displayed the details of each student stored in the array after assigning the namesList and agesList. Lastly, the destructor was called for every object in the array, removing all the student objects.

Table 2-3. Final Driver Program

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1 :

HOA2.1 > Supplementary.cpp > ...

```cpp
#include <iostream>
#include <string>

//Problem 1
class Fruits{
    private:
        std::string fruitsName;
        int fruitsPrice;
        int fruitsQuantity;
    public:
    //constructor
    Fruits(std::string newName = "Blank", int newPrice = 0, int newQuantity = 0){
            fruitsName = std::move(newName);
            fruitsPrice = newPrice;
            fruitsQuantity = newQuantity;
        }
    //deconstructor
    ~Fruits(){
    }
    //copy constructor
    Fruits (const Fruits &copyFruit){
        fruitsName = copyFruit.fruitsName;
        fruitsPrice = copyFruit.fruitsPrice;
        fruitsQuantity = copyFruit.fruitsQuantity;
    }
    //copy assignment operator
    Fruits& operator=(const Fruits& copy){
        if(this != &copy){
            fruitsName = copy.fruitsName;
            fruitsPrice = copy.fruitsPrice;
            fruitsQuantity = copy.fruitsQuantity;
        }
        return *this;
    }
    //sum
    double calculateSum(){
        return fruitsPrice * fruitsQuantity;
    }

    //details
    void printFruits(){
        std::cout << "Fruit Name: " << fruitsName << std::endl;
        std::cout << "Fruit Price: " << fruitsPrice << std::endl;
        std::cout << "Fruit Quantity: " << fruitsQuantity << std::endl;
        std::cout << "" << std::endl;
    }
};
```

```cpp
class Vegetables{
    private:
        std::string vegetableName;
        int vegetablePrice;
        int vegetableQuantity;

    public:
    //constructor
    Vegetables(std::string newName = "JohnPork", int newPrice = 0, int newQuantity = 0){
        vegetableName = std::move(newName);
        vegetablePrice = newPrice;
        vegetableQuantity = newQuantity;
    }

    //deconstructor
    ~Vegetables(){
    }

    //copy constructor
    Vegetables (const Vegetables &copyVegetable){
        vegetableName = copyVegetable.vegetableName;
        vegetablePrice = copyVegetable.vegetablePrice;
        vegetableQuantity = copyVegetable.vegetableQuantity;
    }

    Vegetables& operator=(const Vegetables& copy){
        if(this != &copy){
            vegetableName = copy.vegetableName;
            vegetablePrice = copy.vegetablePrice;
            vegetableQuantity = copy.vegetableQuantity;
        }
        return *this;
    }
```

```cpp
    //sum
    double calculateSum(){
        return vegetablePrice * vegetableQuantity;
    }

    //details
    void printVegetables(){
        std::cout << "Vegetable Name: " << vegetableName << std::endl;
        std::cout << "Vegetable Price: " << vegetablePrice << std::endl;
        std::cout << "Vegetable Quantity: " << vegetableQuantity << std::endl;
        std::cout << "" << std::endl;
    }
};
```

Problem 2:

```cpp
int main(){
    //Problem 2
    const int fvSize = 2;
    double total = 0;

    Fruits* fruitList[fvSize];
    Vegetables* vegetablesList[fvSize];

    std::string fruitsName[fvSize] = {"Apple", "Banana"};
    int fruitPrice[fvSize] = {10, 10};
    int fruitQuantity[fvSize] = {7, 8};

    std::string vegetablesName[fvSize] = {"Broccoli", "Lettuce"};
    int vegetablesPrice[fvSize] = {60, 50};
    int vegetablesQuantity[fvSize] = {12, 10};

    for(int i = 0; i < fvSize; i++){
       fruitList[i] = new Fruits(fruitsName[i], fruitPrice[i], fruitQuantity[i]);
    }

    for(int i = 0; i < fvSize; i++){
        fruitList[i]->printFruits();
    }

    for(int i = 0; i < fvSize; i++){
       vegetablesList[i] = new Vegetables(vegetablesName[i], vegetablesPrice[i], vegetablesQuantity[i]);
    }

    for(int i = 0; i < fvSize; i++){
        vegetablesList[i]->printVegetables();
    }
```

Problem 3:

```cpp
    //Problem 3
    //Fruits
    for(int i = 0; i < fvSize; i++){
        total = total + fruitList[i]->calculateSum();
    }
    //Vegetables
    for(int i = 0; i < fvSize; i++){
        total = total + vegetablesList[i]->calculateSum();
    }

    std::cout << "\nTotal Amount: PHP " << total << std::endl;
```

Problem 4:

```cpp
    //Problem 4
    delete vegetablesList[1];
    vegetablesList[1] = nullptr;

    std::cout << "\nAfter deleting Lettuce:\n";


    for(int i = 0; i < fvSize; i++){
        fruitList[i]->printFruits();
    }

    for(int i = 0; i < fvSize - 1; i++){
        vegetablesList[i]->printVegetables();
    }

    return 0;
}
```

Output:

```
Fruit Name: Apple
Fruit Price: 10
Fruit Quantity: 7

Fruit Name: Banana
Fruit Price: 10
Fruit Quantity: 8

Vegetable Name: Broccoli
Vegetable Price: 60
Vegetable Quantity: 12

Vegetable Name: Lettuce
Vegetable Price: 50
Vegetable Quantity: 10


Total Amount: PHP 1370

After deleting Lettuce:
Fruit Name: Apple
Fruit Price: 10
Fruit Quantity: 7

Fruit Name: Banana
Fruit Price: 10
Fruit Quantity: 8

Vegetable Name: Broccoli
Vegetable Price: 60
Vegetable Quantity: 12

PS C:\Users\Gabriel\Desktop\CPP\HOA2_1>
```

Code Analysis:

The code creates two arrays of pointers, fruitList and vegetablesList, to store the grocery items. Using separate for loops in the driver code, Fruits and Vegetables objects are dynamically created with the new operator, which passes the values from the fruitsName, fruitPrice, fruitQuantity, vegetablesName, vegetablesPrice, and vegetablesQuantity arrays to each class constructors. Inside the constructor, it transfers the value of newName to the object name, while the price and quantity are assigned to their corresponding variables. The address of each allocated object is then stored in the array of pointers. After all the objects are created, separate for loops are used to access each pointer to call the print function to display the name, price, quantity of every bought grocery item. Next, another two for loops calculate the total cost by calling the calculateSum() function for fruits and vegetables. After calculating it returns it to the total variable, so the declared total = 0 would become total = 1370. Finally, the lettuce object is removed from the grocery list by calling the delete vegetablesList[1], which deallocates the dynamically allocated memory, then the ptr was set to nullptr for safety practice. Lastly another for loop is used to display the updated grocery list without the lettuce.

Conclusion :

In this activity, I learned how dynamic memory allocating works by using pointers, arrays, new, delete, and classes. I also understand more how to use constructors, destructors, and copy constructors. Although in the supplementary it was required to make a copy assignment operator I wasn't able to use it and I'm still wondering how it works. The ILO A helped me understand the basics of memory allocation and how objects can be created, copied, stored, and destroyed, which became pretty helpful to me when completing the supplementary activity. The supplementary activity was quite challenging because it required me to apply everything that I learned in this module by creating different classes with constructor, destructor, copy constructor , and using arrays with pointers to store and display the grocery list. Overall, I think I did well in this activity because I felt good that I learned something new and this is what drives me to do my best in every activity. But, I still need to practice more my coding skills when it comes to using pointers, copy constructors, and copy assignment operators.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **ChatGPT was used to properly correct my grammar and punctuation marks.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.